

Einleitung

In meinem Projekt im Rahmen des Drehtürmodells 2023/24 habe ich mich damit befasst, eine Website zu programmieren, welche die Prozesse einer Kryptowährung simuliert. Die Website ist unter <https://kryptosim.eu> zu finden, der Code unter <https://github.com/ToasterXE/kryptosim>.

Warum gerade eine Kryptowährung Gegenstand meines Interesses war, wird sich im Verlauf dieses Dokuments (hoffentlich) zeigen. Betrachten wir herkömmliche Währungen: Sie basieren auf zentralen Banken, die Maßnahmen festlegen können und sich um die Überwachung kümmern. Dies basiert dabei auf Vertrauen (beziehungsweise auf Verträgen, welche jedoch keine einhundertprozentige Sicherheit garantieren) und zieht Gebühren mit sich. Das originale Bitcoin-Paper (<https://bitcoin.org/bitcoin.pdf>) beschäftigt sich mit einem System, welches dezentralisiert ist und ohne benötigtes Vertrauen in etwaige Institutionen Transaktionen korrekt und sicher durchführen kann, also mit der ersten Idee einer Kryptowährung.

Wir beginnen damit, uns eine Liste vorzustellen, in der Transaktionen zwischen Benutzern stehen. An einem gegebenen Punkt können wir die Liste evaluieren und feststellen, welche Benutzer Geld bekommen, und welche Benutzer Geld einzahlen müssen. Wer hierbei wessen Geld bekommt, ist egal. Damit keine Probleme entstehen, verbieten wir jedem Benutzer, mehr auszugeben, als dieser zu dem Zeitpunkt besitzt.

Unser erstes Problem ist, dass Benutzer nicht nur ihre eigenen, sondern auch Transaktionen anderer auf die Liste schreiben können. Es benötigt eine Art der Verifizierung.

LISTE:
Alice gibt Bob 1€
Bob gibt Alice 5€
Charlie gibt Alice 5€
Bob gibt Charlie 6€

Alice +9€
Bob -10€
Charlie +1€

RSA-Schlüsselpaare

Theorie

Wir benutzen eine Art Unterschrift (Signierung), welche uns garantiert, dass der Sender einer Transaktion die Person ist, welche die Transaktion tätigt. Dazu benutzen wir asymmetrische Verschlüsselung. Wir generieren ein Schlüsselpaar folgendermaßen:

1. Wähle zwei zufällige Primzahlen p und q .
 - Es werden zufällige Zahlen generiert und auf Primalität getestet. Dafür wird zunächst der Miller-Rabin-Test verwendet. Da dieser ein probabilistischer Test ist, wird nach Findung einer (wahrscheinlichen) Primzahl noch ein weiterer, ineffizienter Test durchgeführt, welche n auf Teilbarkeit von $(2,3,4 \dots \sqrt{n})$ testet.
2. Berechne das RSA-Modul $N = p * q$ und die Phi-Funktion $\varphi(N) = (p - 1) * (q - 1)$
3. Wähle e als $1 < e < \varphi(N)$, teilerfremd zu $\varphi(N)$
4. Wähle d als multiplikativ modular Inverses zu $e, \varphi(N)$, also $e * d \equiv 1 \pmod{\varphi(N)}$

Man erhält den privaten Schlüssel (d, N) und den öffentlichen Schlüssel (e, N) .

Es gilt nun die Kongruenz: $m \equiv m^{ed} \pmod{N}$.

Mit einem Zwischenschritt erhalten wir: $c \equiv m^d \pmod{N}$ und $m \equiv c^e \pmod{N}$. Hierbei ist m unsere Nachricht im Klartext und c eine verschlüsselte Nachricht. Die oben genannte Kongruenz lässt sich durch den Satz von Euler beweisen, welcher besagt: wenn a und n teilerfremd sind, so gilt: $a^{\varphi(n)} \equiv 1 \pmod{n}$.

e und d sind modular multiplikativ invers $\pmod{\varphi(N)}$, das heißt $e * d = 1 \pmod{\varphi(N)}$, mit anderen Worten, $\varphi(N)$ teilt $ed - 1$.

Wir können ed also schreiben als $ed = 1 + h * \varphi(N)$, woraus folgt:

$$m^{ed} = m^{1+h*\varphi(N)} = m^1 * (m^{\varphi(N)})^h \equiv m * 1^h \equiv m \pmod{N}$$

Also ist für den Fall, dass m und N teilerfremd sind, $m \equiv m^{ed} \pmod{N}$ bewiesen. Für den Fall, dass m ein Teiler von N ist, muss m ein Vielfaches von p oder q sein, da ein Teiler einer Zahl immer ein Vielfaches von einem Primfaktor dieser Zahl ist, und N als Primfaktoren genau p und q besitzt. Laut dem chinesischen Restsatz gilt:

$$(x = y \pmod{p} \wedge x = y \pmod{q}) \rightarrow x = y \pmod{pq}.$$

Wir müssen also zeigen, dass sowohl $m = m^{ed} \pmod{p}$ als auch $m = m^{ed} \pmod{q}$, da dies $m = m^{ed} \pmod{pq = N}$ impliziert.

Wir nehmen an, m ist ein Vielfaches von p . Für den anderen Fall lässt sich der gleiche Beweis mit vertauschten Buchstaben aufstellen. In diesem Fall wissen wir, dass $m \pmod{p} = 0$, weil wir m als Vielfaches von p schreiben können, und die Restmenge bei der euklidischen Division durch p somit leer. m^{ed} ist somit ebenfalls ein Vielfaches von p . Also steht auf beiden Seiten der Gleichung $0 = 0 \pmod{p}$, womit $m = m^{ed} \pmod{p}$ gilt.

Für m und q wissen wir, dass diese teilerfremd sind, da sonst $m = N$ gelten müsste und m nach Definition kleiner als N ist. Also lässt sich wieder der Satz von Euler anwenden. Auch der kleine Satz von Fermat, welcher besagt: $a^{p-1} \equiv 1 \pmod{p}$ für teilerfremde a, p .

$$\begin{aligned} m &= m^{ed} \pmod{q} \\ &= m^{1+h\varphi(N)} \pmod{q} \\ &= m^1 * m^{h(p-1)(q-1)} \pmod{q} \\ &= m^1 * (m^{q-1})^{h(p-1)} \pmod{q} \\ &= m^1 * (1)^{h(p-1)} \pmod{q} \\ &= m * 1 = m \pmod{q} \end{aligned}$$

Also gilt $m = m^{ed} \pmod{q}$ und $m = m^{ed} \pmod{p}$, und somit $m = m^{ed} \pmod{N}$.

Mit diesen Schlüsseln können wir nun Nachrichten verifizieren. Der Sender einer Nachricht kann diese vor dem Senden mit seinem privaten Schlüssel verschlüsseln. Andere können diese dann mit dem öffentlichen Schlüssel entschlüsseln, und so verifizieren, dass der Sender der Nachricht den zu dem öffentlichen Schlüssel gehörenden privaten Schlüssel besitzen muss. Verschlüsselte Transaktionen können nicht einfach kopiert und erneut gesendet werden, da sie eine einmalige ID besitzen, die auch verschlüsselt wird. Die Schlüssel können auch in die andere Richtung

genutzt werden, sodass eine Nachricht gezielt an einen Empfänger geschickt wird, indem sie mit dessen öffentlichem Schlüssel entschlüsselt wird und nur noch mit dem privaten Schlüssel entschlüsselt werden kann. Natürlich ist es ebenfalls möglich, eine Nachricht mehrmals, mit mehreren Schlüsseln zu verschlüsseln (z.B. vom Sender signiert und für den Empfänger verschlüsselt). Die Verschlüsselung von Nachrichten garantiert Privatsphäre und dass die Nachricht vom Sender zum Empfänger keinen Veränderungen untergangen ist.

Praxis

Dieses System wird im Großen und Ganzen implementiert, wie oben beschrieben. Dem aufmerksamen Leser ist eventuell aufgefallen, dass man den privaten Schlüssel einer Person berechnen könnte, wenn man den öffentlichen Schlüssel kennt ($e = \text{Inverses zu } d \bmod \varphi(N)$), da die Zeitkomplexität der Berechnung von multiplikativ Inversen nur $\mathcal{O}(\log(n))$ mit n als Modul beträgt. Die Komplexität der Berechnung von $\varphi(N)$ beträgt jedoch $\mathcal{O}(\text{sqrt}(n))$, wodurch die Phi-Funktion bei großen Zahlen ($n > 2^{256}$) nicht mehr effizient zu berechnen ist, wenn man die ursprünglichen Werte für p und q nicht besitzt. Man braucht also sehr große Schlüssel, damit diese wirklich sicher sind.

Es existiert außerdem eine weitere Schwachstelle der Umsetzung: Wenn zwei ähnliche Nachrichten mit dem gleichen Schlüssel verschlüsselt werden, ähneln sich auch die verschlüsselten Nachrichten, da dadurch, dass die Schlüssel in der Umsetzung relativ klein sind, keine besonders langen Zeichenketten auf einmal verschlüsselt werden können, sondern immer nur kürzere Abschnitte der vollständigen Eingabe.

Hierzu gibt es einen einfachen Lösungsansatz, der in Zukunft implementiert werden soll. Die Nachrichten sollen vor der richtigen Verschlüsselung mit einem zufälligen Schlüssel vorverschlüsselt werden. Das heißt, die Nachrichten werden zu einer scheinbar zufälligen Zahl, die auf externen Faktoren basiert. Konkret soll die Nachricht mit einer aus Datum und Uhrzeit hervorgehenden Zahl verschlüsselt werden. Das Datum wird dabei im Klartext mitgespeichert, sodass die Nachricht wieder entschlüsselt werden kann. Bei der Entschlüsselung wird die Nachricht zunächst mit dem gegebenen RSA-Schlüssel entschlüsselt, danach mit dem zu N modular multiplikativ Inversen zu dem aus dem Datum herausgehenden Schlüssel erneut entschlüsselt. Die Schlüsseldaten eines Datums sind zwar nicht geheim, sind jedoch immer anders und sorgen so dafür, dass zwei identische, an verschiedenen Zeitpunkten verschlüsselte Nachrichten nach ihrer Verschlüsselung verschieden aussehen, und wieder zu identischen Nachrichten entschlüsselt werden können. Es gibt nun keine Möglichkeit mehr, mithilfe der verschlüsselten Nachricht (bzw. Datenmengen von verschlüsselten Nachrichten) auf die ursprüngliche Nachricht zu schließen.

Knoten

Kommen wir zurück zu unserer Liste. Wir können nun also verifizieren, von wem Transaktionen auf die Liste geschrieben wurden. Unser nächstes Problem ist, dass die Liste in ihrer jetzigen Form zentralisiert ist. Um dies zu beheben, geben wir jedem Benutzer eine eigene Kopie der Liste. Benutzer können jetzt, um Einträge auf der Liste anzufertigen, die Transaktionsnachricht in das Netzwerk senden. Alle Instanzen des Netzwerks warten darauf, Transaktionen zugeschickt zu bekommen. An dieser Stelle müssen Knoten eine Möglichkeit haben, sicherzustellen, dass sie die gleichen Informationen besitzen wie alle anderen, um Betrug und Falschinformationen zu vermeiden.

Alices LISTE: Alice gibt Bob 1€ Bob gibt Alice 5€ Charlie gibt Alice 5€ Bob gibt Charlie 6€ Charlie gibt Alice 9€	Bobs LISTE: Alice gibt Bob 1€ Bob gibt Alice 5€ Charlie gibt Alice 5€ Bob gibt Charlie 6€	Charlies LISTE: Alice gibt Bob 1€ Bob gibt Alice 5€ Charlie gibt Alice 5€ Bob gibt Charlie 6€
---	--	--

Abb. 2: Alice kann nicht wissen, dass sie im restlichen Netzwerk keine Überweisung von Charlie erhalten hat.

Blockchain

Dazu gibt es folgendes Konzept: Jede neue Transaktion muss eine Referenz zu der vorherigen Transaktion enthalten, und zwar einen Hash der vorherigen Transaktion. Ein Hash ist eine Art irreversible Verschlüsselung einer Eingabe, bei welcher das Ergebnis (hier) immer eine feste Größe hat und sich bei einer kleinen Änderung in der Eingabe komplett verändert. Dieser wirkt sich dann auch auf den Hash der neuen Transaktion aus, sodass eine Kette von Transaktionen entsteht.

Zudem enthält jede gesendete Transaktion einen Beweis für Rechenleistung, das heißt, es muss vor dem Senden einer Transaktion ein rechenaufwendiges Rätsel gelöst werden, dessen Korrektheit sich mit wenig Rechenleistung überprüfen lässt. Dieser Rechenbeweis heißt „Proof of Work“ und der Prozess des Berechnens „Mining“. Sender von Transaktionen müssen diese also *minen*, bevor sie sie in das Netzwerk schicken. Mehrere Miner versuchen gleichzeitig, einen *Proof of Work* für (meist) verschiedene Transaktionen zu berechnen. Ist einer von ihnen erfolgreich, so müssen die anderen von vorne anfangen, da sich der Hash der letzten Transaktion geändert hat, welcher Teil der Berechnungen ist. Wenn Bob und Charlie jetzt gleichzeitig eine Transaktion (=beide Transaktionen haben die gleiche vorherige Transaktion) an Alice senden, Charlie seine Transaktion jedoch nur an Alice gesendet hat, und nicht an das gesamte Netzwerk, so hat Alice zunächst keinen Weg, zu wissen, welcher Transaktion sie vertrauen soll. Daher werden beide Ketten beibehalten, bis Eine deutlich länger ist als die andere. Damit Charlie dieses Rennen gewinnt, müsste er für längere Zeit schneller neue Transaktionen minen als der Rest des Netzwerkes, welcher zusammen an einer Kette arbeitet. Praktisch braucht er dafür also mehr als die Hälfte der gesamten Rechenleistung im System.

Da dies bei einem Netzwerk mit vielen Knoten unmöglich ist, ist es insgesamt nicht möglich, einzelne Knoten gezielt zu manipulieren.

Aus Effizienzgründen berechnen wir keine einzelnen Transaktionen, sondern mehrere (hier 3) Transaktionen in einem „Block“. Nicht jeder Sender einer Transaktion muss diese selbst in einem

Block minen. Stattdessen wird die Transaktion an alle Knoten geschickt, von diesen jeweils verifiziert und in einen „Pool“ wartender Transaktionen aufgenommen, aus dem ein Miner sich beliebige Transaktionen aussuchen kann.

Der Miner eines Blockes erhält für die Rechenleistung eine Belohnung („Miners Reward“) in Form von Währung. Diese Belohnung wird von niemandem gezahlt, sondern es entsteht dabei neue Währung. So kann es auch funktionieren, dass von Beginn der Blockchain an nie in einer Transaktion mehr ausgegeben wird, als der Sender besitzt. Der erste Block der Blockchain enthält so keine Transaktionen, außer dem *Miners Reward*. Damit es nicht unendlich viel Währung gibt, wird der Betrag, den die Miner erhalten, immer in einem bestimmten Zeitabstand halbiert, sodass eine Maximalsumme für die Währung entsteht und die Inflationsrate der Währung sich mit der Zeit stabilisiert. Bei Bitcoin zum Beispiel war der ursprüngliche Miners Reward 50 Bitcoin und wird alle 210.000 Blöcke halbiert. Dies wird ungefähr alle vier Jahre erreicht und fand zuletzt am 20. April 2024 statt. Bild 1 zeigt, wie sich Bitcoin über die Zeit stabilisiert hat und sich in Zukunft weiter stabilisieren wird.

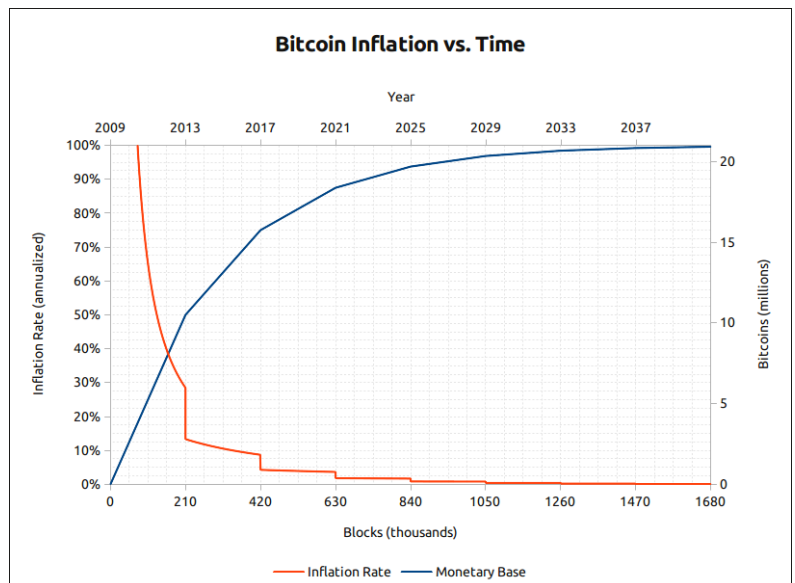


Bild 1: Jährliche Inflationsrate und Gesamtsumme der Währung von Bitcoin
<https://www.bitcoinblockhalf.com/images/bitcoin-inflation-chart.png>

Praxis:

1. Berechnen von dem Hash eines Blockes

Es wird die sha256-Hashfunktion verwendet. Diese nimmt eine Eingabe und berechnet daraus deterministisch einen 256-Bit-langen Hash, welcher nicht zurück zur ursprünglichen Eingabe konvertiert werden kann. Jedenfalls wurde weltweit von Informatikern und Mathematikern noch kein Ansatz gefunden, dies zu tun. Bewiesen, dass es unmöglich ist, wurde jedoch auch nicht. Auf jeden Fall gilt diese Hashfunktion als eine sichere und weit benutzte Verschlüsselungsfunktion (z.B. bei Ihren Emails). Die Eingabe für einen Block lautet:

$$\text{hash} = \text{sha256}(\text{string}(\text{prevhash}, t1, t2, t3, \text{rewardtransaction}, \text{pow}));$$

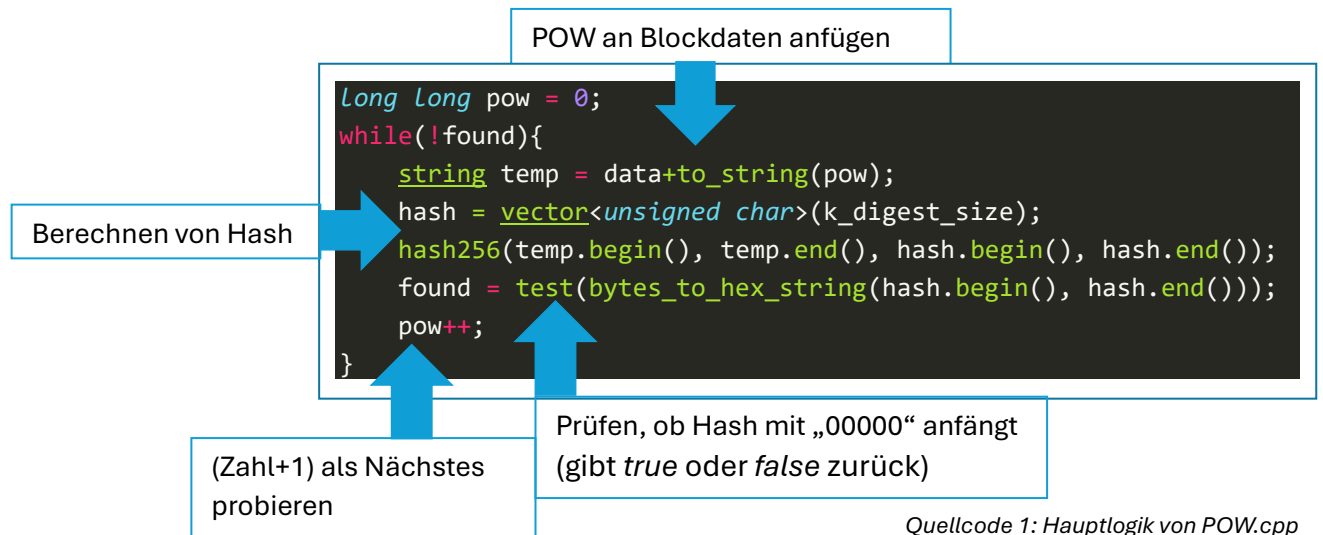
Der Hash eines Blockes wird im Hexadezimalsystem angegeben, um ihn auf $\log_{16}(2^{256}) = 64$ Stellen zu komprimieren.

2. Berechnen des POWs

Der Proof of Work ist eine Zahl, die am Ende des Blockes angehängt wird und mit in die Hashfunktion gegeben wird. Dabei ist eine Zahl gesucht, die dafür sorgt, dass der Hash des Blockes mit einer bestimmten Anzahl (hier 6) von Nullen anfängt. Da es mathematisch keinen Weg gibt, eine solche Zahl effizient zu berechnen, und die Wahrscheinlichkeit für einen solchen Hash bei 1 zu $16^6 = 16.777.216$ liegt, muss entsprechend viel Rechenleistung erbracht werden.

Daher kommen die Begriffe „Proof of Work“ und „Mining“. Die Berechnung des POWs dauert mit 6 gewünschten Nullen am Anfang und dem Programm POW.exe, welches auf der Startseite der Website unter dem Abschnitt „Downloads“ zu finden ist, wenige Minuten (~10, kommt auf Rechner an).

Dieses Programm enthält eine einfache Brute Force Implementierung.



In der Logik gibt es wie bereits erwähnt keine Optimierungsmöglichkeiten. Jedoch ist es möglich, die Hardware eines Rechners besser auszunutzen, in dem man entweder den Prozess auf allen Prozessorkernen gleichzeitig laufen lässt (und hierbei Überschneidungen der zu testenden Zahlen vermeidet), oder die Prozesse auf die Grafikkarte verschiebt. Grafikkarten sind darauf ausgelegt, Shaderprogramme auszuführen. Shader führen viele Berechnungen parallel durch (siehe Drehtürmodell 2022/23 „3D-Grafik und OpenGL“). Es kann dabei deutlich mehr parallel berechnet werden als auf einem Prozessor. Besonders ambitionierte Miner manipulieren also Grafikkarten dahingehend, anstatt Farbwerten für viele Pixel, Hashes für viele Zahlen zu berechnen und dabei einen validen POW zu finden. Dies ist allerdings nicht vorteilhaft für die Hardware und den Grafikkarten Markt.

Zufällige Zahlen als Testzahl für den POW auszuwählen, ist keine Optimierung, da mathematisch die gleiche Chance besteht, eine valide Zahl zu finden, wenn man systematisch Zahlen hochzählt.

3. Netzwerkknoten

Netzwerkknoten sind wie bereits erwähnt einzelne Instanzen im Netzwerk, die ihre eigene Kopie der Blockchain besitzen und eingehende Transaktionen und Blöcke verifizieren. Optional können sie die Fähigkeit besitzen, selbst Transaktionen und Blöcke in das Netzwerk zu senden. Genau genommen besteht das Netzwerk nur aus Knoten. Dieser Begriff kommt von der Datenstruktur des Graphen, welche aus verschiedenen Knoten besteht, die durch (un)gerichtete, (un)gewichtete Kanten miteinander verbunden sind. Eine spezielle Art von Graphen sind Bäume, welche azyklisch sind und in welchen jeder Knoten außer der Wurzel einen Eingangsgrad von genau 1 hat.

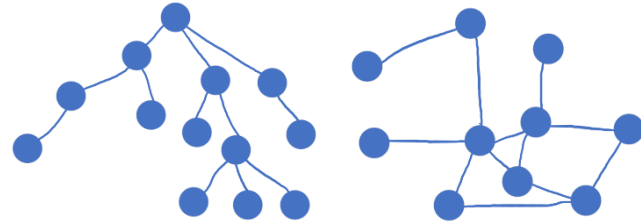


Abb. 3: Graph (rechts) und Baum (links)

In meiner Simulation habe ich ein Programm für Netzwerkknoten in Python implementiert, da diese Programmiersprache sich aufgrund von intuitiv nutzbaren Bibliotheken anbot. Ein Netzwerkknoten sendet alle 5 Sekunden eine Anfrage an den Webserver, auf einer Seite, die für normale Benutzer nicht sichtbar ist und für die Kommunikation mit den Netzwerkknoten zuständig ist. Vorher, beim Starten des Knotenprogramms, meldet der Knoten sich bei dem Webserver an und bekommt eine ID zurück, welche er bei seiner Anfrage an den Server schickt. Der Server ruft dann die Zeile in der *nodes* Datenbank auf, welche der gesendeten ID entspricht und aktualisiert einen Wert *last_seen*, welcher den Zeitpunkt des letzten „Pings“ eines Knoten an den Server enthält. Der Server prüft außerdem, ob eine Transaktion oder ein Block auf Verifizierung von diesem Knoten wartet. In diesem Fall ruft er aus einer anderen Tabelle der Datenbank die Daten der Transaktion/des Blockes ab und schickt diese an den Knoten zurück. Der Knoten verarbeitet dann diese Daten, indem er sie auf Korrektheit prüft (siehe Kriterien unten) und dementsprechend eine Rückmeldung an den Server sendet. Blöcke kommunizieren nicht direkt miteinander, sondern immer nur über den Server, weil dies die Implementierung sowie die Installation bzw. das Hosten eines Netzwerkknotens erleichtert.

Die Verifizierungskriterien für eine Transaktion sind:

- Vom Sender der Transaktion signiert
- Sender gibt nicht mehr aus, als er hat

Für einen Block gibt es mehr Kriterien:

- Vorheriger Hash entspricht „header“ von neuem Block
- Alle Transaktionen sind korrekt (erneute Verifizierung)
- Im Block berechneter Hash entspricht tatsächlichem Hash
- Hash fängt an mit „000000“

Schlägt die Verifizierung eines Blockes fehl, synchronisiert der Knoten seine Blockchain mit dem restlichen Netzwerk, indem er über eine Anfrage an den Webserver die gesamte Blockchain jedes aktiven Knotens herunterlädt und die am häufigsten vorkommende Blockchain, deren Länge der der eigenen Blockchain entspricht (andernfalls kann der Knoten vorzeitig einen Block aufnehmen, den er später noch über das Netzwerk erhält und somit dann als inkorrekt empfindet) als eigene Blockchain nimmt. Ist ein Block immernoch inkorrekt, so bedeutet dies, dass das gesamte Netzwerk diesen Block für inkorrekt hält, und der Knoten gibt an den Server zurück, dass der Block verworfen werden soll. Verifiziert ein Knoten erfolgreich eine Transaktion

oder einen Block, gibt er an den Server zurück, dass die Daten an den nächsten Knoten zur Verifizierung gesendet werden können. Der Server sucht dann einen Knoten mit einer höheren ID, welcher in den letzten paar Sekunden einen Ping gesendet hat. Existiert kein solcher Knoten, heißt das, dass die Daten vollständig verifiziert wurden, und dies wird entsprechend in die Datenbank eingetragen.

Immer wenn ein Knoten seine Blockchain aktualisiert, sendet er sie an den Server. Man notiere, dass das Synchronisieren einer Blockchain nur in dem Fall relevant ist, in dem die Blockchain lokal an einem Knoten verändert wurde, da dies die einzige mögliche Fehlerquelle ist. Dieser Vorgang ist also eine reine Sicherheitsmaßnahme, ersetzt jedoch das Prinzip, alle empfangenen Blöcke zu behalten, bis eine eindeutig längere Teilkette existiert. In großen Kryptowährungen passiert diese Synchronisierung nicht, weil das Netzwerk zu groß ist, um dies bewerkstelligen zu können. Zwar können effiziente Algorithmen leicht mit Graphen mit Knotenmengen von einigen Millionen umgehen, jedoch sind auch die zu sendenden Daten einfach zu groß (eine Blockchain wird leicht einige Gigabyte groß). In dem Fall eines überschaubaren Netzwerkes ist eine Synchronisierung sinnvoller.

Das Knotenprogramm ist auch auf der Website verfügbar, recht simpel einzurichten und nicht sehr ressourcenbeanspruchend.

Analyse der Umsetzung

Daten liegen auf dem Server

Man könnte argumentieren, das System sei nicht dezentralisiert, weil sich die gesamten Daten auf der Website auf einem Server befinden, so auch die Datenbank mit privaten und öffentlichen Schlüsseln. Dabei muss man allerdings differenzieren zwischen dem System und der Website bzw. der Benutzeroberfläche, die dieses zugänglich macht. Bei Kryptowährungen wie Bitcoin werden Transaktionen üblicherweise ebenfalls nicht manuell über von einem eigenen Knoten aus gesendet, sondern es gibt verschiedene Anbieter von Oberflächen, über die Accounts (Wallets) einfach verwaltet werden können. Diese Oberflächen speichern teilweise auch die privaten Schlüssel für den Benutzer ab, damit dieser ihn nicht vergisst, so wie hier. Des Weiteren ist es auch möglich, Transaktionen zu tätigen, ohne dass sich ein privater Schlüssel in der Datenbank befindet. Hierzu müsste man sich außerhalb der Website ein Schlüsselpaar generieren und z.B. durch Mining eine einkommende Transaktion auf den öffentlichen Schlüssel erhalten. Insgesamt wird die Dezentralisierung hierdurch also nicht eingeschränkt.

RSA-Verschlüsselung

Wie zuvor beschrieben, funktioniert das Verschlüsselungssystem korrekt. Ein Problem in der Umsetzung ist jedoch, dass sehr kleine Schlüssel generiert werden. Das heißt, es ist möglich, von einem öffentlichen Schlüssel ausgehend den privaten Schlüssel zu berechnen. Dies ist normalerweise dadurch schwer, dass p und q sehr große Zahlen sind ($\geq 2^{256}$) und $\varphi(N)$ dadurch schwer zu berechnen ist. Da ich die Schlüsselgenerierung selbst programmiert habe, konnte ich nicht gegen die Ganzzahllimits der Sprache *php* ankommen und musste kleinere

Schlüssel verwenden. Es ist geplant, größere Schlüssel zu implementieren, eventuell dadurch, dass die Schlüssel nicht mehr als Zahl behandelt werden, sondern als Zeichenkette, wobei Operationen wie Plus und Minus „von Hand“ implementiert werden müssten.

Verifizierung im Netzwerk

Die Verifizierung im Netzwerk verläuft sinnvoll. Es ist nicht möglich, dass eine Transaktion fälschlicherweise verifiziert wird, jedoch könnte es passieren, dass eine Transaktion fälschlicherweise nicht verifiziert wird, was jedoch keine Fehler im weiteren System verursacht, da es lediglich dazu führt, dass eine weitere Verifizierungsanfrage gesendet werden muss. Das Netzwerk kann durch fehlerhafte bzw. sporadisch inaktive Knoten gestört werden, welche den oben genannten Fehler verursachen. Ein zweiter Nachteil ist, dass die Verifizierung im Netzwerk etwa $4s \cdot (\text{Anzahl Knoten})$ benötigt, da diese jeweils nur alle vier Sekunden mit dem Server kommunizieren und im schlechtesten Fall alle zum gleichen Zeitpunkt eine Anfrage senden. Es könnte das Anfrageintervall verkürzt werden.

Korrektheit der Blockchain

Hier gibt es keine (bekannten) Schwachstellen. Die Blockchain konnte bisher durch verschiedenste Ansätze nie erfolgreich manipuliert werden.

Verifizierung von Blöcken

Dieser Aspekt wurde bereits diskutiert. Hierbei wird bei dem Auftreten eines fehlerhaften Blocks die Blockchain des Knotens mit dem Netzwerk synchronisiert. Wenn also *Charlie* versucht, einen Block nur an *Alice* zu senden, so kann ihm dies durchaus gelingen. Wenn *Alice* jedoch einen Block des korrekten Netzwerkes erhält, wird dieser als fehlerhaft erkannt und die Blockchain wird synchronisiert, wodurch *Charlies* Block verworfen wird. Da in der Simulation das Netzwerk nicht besonders groß ist, ist diese etwas ineffizientere Variante genügend.

Ist das System dezentralisiert?

Kommen wir zum Schluss zu der wohl interessantesten Frage. Zunächst wollen wir klären, was erfüllt sein muss, damit eine Dezentralisierung vorliegt:

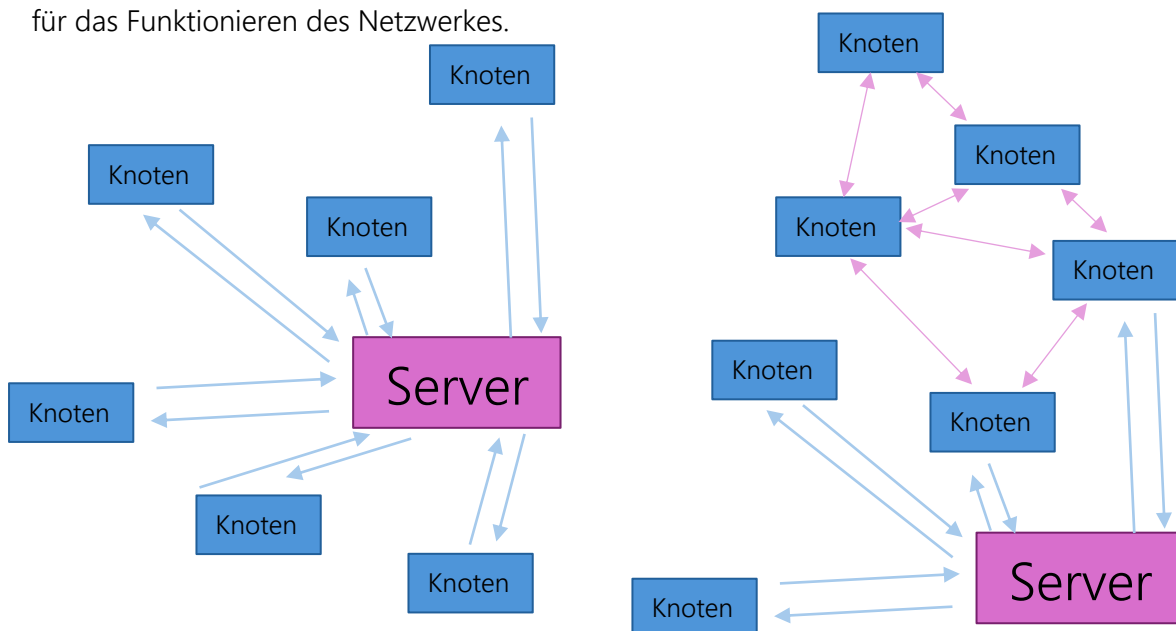
1. Es existieren voneinander unabhängige Einheiten
2. Alle Einheiten müssen die gleichen Fähigkeiten haben
3. Keine Einheit darf essenziell sein, das heißt das System muss ohne die Einheit funktionieren können

Der erste Aspekt ist recht simpel, in unserem System existieren voneinander unabhängige Netzwerkknotten, also ist diese Bedingung erfüllt.

Wie unschwer zu erkennen ist, erbringt der Server sehr viel mehr als die Netzwerkknotten. Die Möglichkeit, sich unabhängig vom Netzwerk ein Schlüsselpaar zu generieren und dieses zusammen mit Login-Daten in einer Datenbank zu speichern, kann auf jedem Knoten implementiert werden. Ferner können alle weiteren Funktionen auch auf jedem anderen Knoten implementiert werden, da diese alle auf Daten aufbauen, welche alle Knoten besitzen, nämlich der Blockchain und dem Transaktionenpool. Knoten müssten hier zusätzlich noch speichern, welche Transaktionen verifiziert sind. Auch die Fähigkeit, Transaktionen und Blöcke zu senden, kann für Knoten implementiert werden, da die Server-Seite dieses Vorgangs bereits

programmiert wurde. Insgesamt könnte man die gesamte Website also auch von einem beliebigen Knoten aus aufbauen.

Zu dem Dritten Aspekt fällt nun auf, dass die Knoten des Netzwerkes nur über den Server miteinander kommunizieren. Ohne den Server könnten die Knoten also nicht mehr kommunizieren, wodurch die dritte Bedingung nicht erfüllt wäre. Allerdings lässt sich dies umgehen. Das jetzige Knotenprogramm wurde aus Zeitgründen möglichst simpel gehalten. Dieses ist jedoch nicht benötigt, um einen Netzknoten zu besitzen. Das heißt, Benutzer können selbst Knoten programmieren (oder auch nur das existierende Programm abändern), da der Server mit jeder Anfrage kommuniziert und auch öffentlich bekannt ist, wie die Anfragen ablaufen (welche Daten müssen wohin geschickt werden für welche Rückmeldung). In dem jetzigen System (links) ist es möglich, das System durch direkt miteinander kommunizierende Knoten zu dezentralisieren. Der Server kann weiterhin existieren, ist jedoch keine Voraussetzung für das Funktionieren des Netzwerkes.



Natürlich kann man nicht unbedingt behaupten, ein System sei dezentralisiert, weil es von jeder Person zu einem dezentralisierten System gemacht werden könnte, aber ein System kann auch nicht zentralisiert sein, wenn die zentrale Einheit keine Macht über andere Einheiten besitzt, nichtmal die, die zentrale Einheit zu bleiben.

Deswegen würde ich das System insgesamt als sinnvoll und durchdacht bezeichnen, da es den Zielvorgang wiedergibt und bis auf ein paar Sicherheitsrisiken mathematisch korrekt funktioniert. Es zeigt, wie Transaktionen ohne zentrale Einheiten stattfinden können. Man vertraut dabei nicht auf Verträge, sondern auf die brillante Korrektheit der Mathematik.